

ETIS

Azure Operations CLI User Guide

Complete operator reference for production inspection, validation, troubleshooting, and guarded runtime scaling

v0.18.0 release manual edition. Production acceptance has separate source and deployment provenance.

Audience	ETIS production operators, course owners, maintainers, and administrators
Purpose	Use the supported command-line interface safely, interpret every result, and know what to do when production differs from the accepted baseline.
Production service	https://simulator.etisframework.org
Reference implementation	COMP 330/474 Software Engineering • Fall 2026
Version	v0.18.0 • October 2, 2026

OPERATING PRINCIPLE

Production changes must be deliberate, reversible where possible, observable, and validated. The repository and protected deployment workflow remain authoritative; local tools do not replace controlled deployment.

Contents

Section	Purpose
Executive Overview	Operating philosophy, accepted baseline, and command selection

Section	Purpose
1-3	Safety model, prerequisites, output semantics

Section	Purpose
4	Complete command reference with all options and examples

Section	Purpose
5-7	Normal operating workflows, troubleshooting, and escalation

Section	Purpose
Appendices	Command matrix, expected baseline, exit behavior, and examples

Executive Overview

The ETIS Azure Operations CLI is the supported operator interface for inspecting ETIS Engineering Studio production. It converts raw Azure state into explicit PASS, WARN, FAIL, and INFO results and keeps routine operations read-only. It is deliberately not a deployment tool.

Operator command map

Use the narrowest command that answers the operational question.

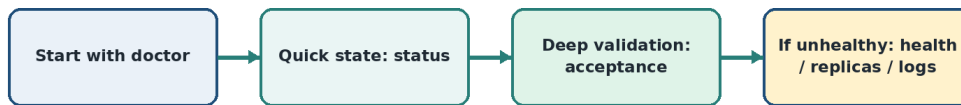


Figure 1. Operator command map. Start with readiness, move to concise state, then run full acceptance when the question is “is this deployment production-ready?” Historical reference figure; current text governs.

ACCEPTED PRODUCTION BASELINE

Production application/resource group: etis-studio-prod; canonical URL: <https://simulator.etisframework.org>; revision mode: Single; target port: 8000; external ingress enabled; min/max replicas: 1 / 5; normal semester posture keeps one replica warm.

Question	Preferred command
Can this workstation safely operate ETIS?	doctor
Is production generally healthy right now?	status
What is the live non-secret configuration?	config
Are public health/readiness endpoints healthy?	health
Are replicas ready/restarting?	replicas
Which revision has traffic?	revisions
What happened recently?	logs
What is month-to-date spend?	cost
Are budget controls configured?	budget
Does the externally visible app behave correctly?	smoke

Question	Preferred command
Does runtime match the accepted/source baseline?	drift
Did the deployment pass all required production checks?	acceptance
Do I need a temporary runtime scaling override?	scale - guarded mutation

1. Operating model and safety boundaries

- Read-only commands are the normal path.
- `scale` is the only current command that mutates live Azure runtime configuration.
- Production deployment remains branch → PR → CI → merge → protected GitHub Deploy Azure workflow.
- Secrets, tokens, Key Vault secret values, and database credentials are intentionally not printed.
- Cost telemetry is advisory and may lag or be throttled without making Studio unhealthy.
- A drift finding is a reason to investigate, not permission to “fix” Azure blindly.

2. Prerequisites and first use

1. Open a terminal in the `etis-engineering-studio` repository.
2. Activate the Python environment used by the repository if applicable.
3. Confirm Azure CLI authentication and subscription context.
4. Run the local readiness check before any production investigation.

```
./scripts/azure/etis-azure doctor
```

FIRST-TIME OPERATOR

If `doctor` does not show the intended subscription, resource group, and production Container App, stop before using `scale` or making any portal changes.

3. Reading output

State	Meaning	Operator response
PASS	The checked invariant matches the accepted/healthy condition.	Continue. A collection of PASS results supports, but does not replace, the final acceptance report.
WARN	The command completed but found an advisory or degraded condition.	Read the note. Determine whether it is expected, transient, or a precursor to failure.
FAIL	A required invariant or operation failed.	Do not declare production accepted. Investigate the indicated boundary before mutation.
INFO	Context or telemetry without pass/fail authority.	Use it to understand the environment; do not treat it as an acceptance gate.

4. Complete command reference

4.1 `help`

Purpose. Display the command catalog and command-specific option pointers.

Examples

```
./scripts/azure/etis-azure help  
./scripts/azure/etis-azure -h  
./scripts/azure/etis-azure --help
```

What you are looking for

Help must render without Azure authentication. `scale` must be classified as mutating. Deployment must remain explicitly outside this CLI.

If it does not match

If help requests Azure login, local argument parsing has regressed. Fix the CLI/tests before relying on it.

4.2 `doctor`

Purpose. Check workstation tools, repository context, Azure authentication/subscription, Container App access, and source replica baseline.

Examples

```
./scripts/azure/etis-azure doctor
```

What you are looking for

Azure CLI/Git/curl available; repo recognized; Azure authentication PASS; production Container App accessible; Bicep baseline 1/5. A dirty working tree may WARN.

If it does not match

Correct missing prerequisites or Azure context. If Bicep differs, inspect the source change before any runtime action.

4.3 `status`

Purpose. Show a concise production snapshot.

Examples

```
./scripts/azure/etis-azure status
```

What you are looking for

PRODUCTION; app/resource group etis-studio-prod; provisioning Succeeded; Single revision mode; 100% traffic to expected revision; min/max 1/5; at least one running replica; STATUS: PASS.

If it does not match

Use `config`, `revisions`, `replicas`, and `drift` to isolate the mismatch. Wrong subscription/app context is a stop condition.

4.4 `config`

Purpose. Display important non-secret runtime configuration and source baseline.

Examples

```
./scripts/azure/etis-azure config
```

What you are looking for

External ingress Yes; target port 8000; Single revision mode; runtime 1/5; Bicep 1/5; secret values intentionally absent.

If it does not match

Treat ingress/port/revision-mode differences as material drift. Determine whether a temporary scale override exists before changing anything.

4.5 `health`

Purpose. Check public `/health`, `/ready`, migration-current state, root reachability, and latency.

Examples

```
./scripts/azure/etis-azure health
```

What you are looking for

`/health` 200 status=ok environment=production; `/ready` 200 status=ready migration_current=True; root reachable; HEALTH: PASS.

If it does not match

If `/health` fails, inspect replicas/ingress/logs. If `/ready` fails, prioritize DB connectivity/migration state. `migration\_current=False` blocks acceptance.

4.6 `replicas`

Purpose. Inspect configured capacity, replica readiness, restart count, revision ownership, and running state.

Examples

```
./scripts/azure/etis-azure replicas
```

What you are looking for

Configured 1/5; one or more ready/running replicas; restart counts stable/low; current revision expected; REPLICAS: PASS.

If it does not match

Zero ready replicas or repeated restarts: run `logs`, `health`, and check the Container App alert/revision. Avoid scaling as a substitute for fixing startup failure.

4.7 `revisions`

Purpose. Inspect revision history and traffic allocation.

Examples

```
./scripts/azure/etis-azure revisions
```

What you are looking for

Current/latest revision identifiable; active; 100% traffic under Single mode; replica count expected.

If it does not match

Unexpected traffic or revision: compare deployment workflow/run SHA and `status`. Resolve deployment state before acceptance.

4.8 `logs`

Purpose. Show structured recent console activity while suppressing routine probes by default.

Examples

```
./scripts/azure/etis-azure logs  
./scripts/azure/etis-azure logs --tail 100  
./scripts/azure/etis-azure logs --all  
./scripts/azure/etis-azure logs --raw  
./scripts/azure/etis-azure logs --follow  
./scripts/azure/etis-azure logs -f
```

What you are looking for

Default view emphasizes real requests/errors and summarizes health/readiness probes. `--all` includes probes; `--raw` removes formatting; `--follow/-f` streams live logs; `--tail N` changes history depth.

If it does not match

HTTP/application error counts should drive investigation. Raw/follow are escalation tools; stop streaming with Ctrl-C.

4.9 `cost`

Purpose. Show month-to-date Azure cost telemetry.

Examples

```
./scripts/azure/etis-azure cost
```

What you are looking for

Cost telemetry returns when available. If Azure Cost Management throttles (429), COST: WARN is advisory and exit remains non-blocking.

If it does not match

Do not hammer the API after 429. Wait and retry later. Use budget alerts as the durable stewardship control.

4.10 `budget`

Purpose. Verify the production budget and thresholds without printing recipient email addresses.

Examples

```
./scripts/azure/etis-azure budget
```

What you are looking for

\$100 monthly budget; 50/80/100% actual-cost alerts enabled; at least one recipient; BUDGET: PASS.

If it does not match

If missing or changed, restore the intended budget configuration or document the approved new baseline. Budget issues do not mean the app is technically down.

4.11 `smoke`

Purpose. Run non-destructive external production checks including authentication/bootstrap invariants.

Examples

```
./scripts/azure/etis-azure smoke
```

What you are looking for

Health/readiness 200; production environment; migrations current; root reachable; Loyola login marker present; native fail-closed shell marker present; Entra route redirects to Microsoft; SMOKE TEST: PASS.

If it does not match

A smoke failure is externally observable. Treat auth/bootstrap failures as release-blocking even when `/health` is green.

4.12 `drift`

Purpose. Compare runtime state with source-of-truth and accepted production invariants.

Examples

```
./scripts/azure/etis-azure drift
```

What you are looking for

App/resource group, Bicep 1/5, runtime 1/5, Single revision mode, external ingress, port 8000, 100% traffic, fail-closed shell all PASS.

If it does not match

Do not auto-correct. Determine whether runtime or source is wrong. A temporary live override should be documented and eventually reconciled through source/deployment.

4.13 `acceptance`

Purpose. Orchestrate the required post-deployment checks and issue the production decision.

Examples

```
./scripts/azure/etis-azure acceptance
```

What you are looking for

Required: status, health, replicas, smoke, drift, budget PASS. Cost is informational. Final line: PRODUCTION ACCEPTANCE: PASS.

If it does not match

Any required FAIL blocks acceptance. Investigate the failing subcommand. Advisory cost WARN must not be misrepresented as production failure.

4.14 `scale`

Purpose. Apply a guarded live runtime min/max replica override.

Examples

```
./scripts/azure/etis-azure scale --help
./scripts/azure/etis-azure scale 1 5
./scripts/azure/etis-azure scale 0 5
```

What you are looking for

Invalid ranges are rejected before Azure authentication. Maximum above 10 is rejected. A no-op exits without mutation. A real change shows target/current/requested values, warns about Bicep authority, and requires typing `PROD`.

If it does not match

Use only for deliberate temporary runtime control. `0 5` re-enables scale-to-zero/cold-start risk. Permanent changes belong in Bicep and the protected deployment path.

Release version and live provenance

The v0.18.0 source version is defined in apps/api/app/version.py. FastAPI/OpenAPI and /health use that value. Source version, merged Git SHA, immutable image/revision, deployment success and production acceptance are separate evidence. The August GO records and v0.17 analytical history keep their original dates and versions.

Accepted production uses shadow/shadow evaluation with legacy student-visible authority. Shadow validator/planner/selector/realizer evaluation remains internal and does not replace student-visible authority. This release introduces no PR5 authority transfer, database migration or course-model change.

A CLI health PASS alone does not verify the expected release image or the student journey. Compare /health version and modes with the intended deployment SHA and immutable image; then run acceptance and the human review/evidence smoke described in docs/releases/v0.18.0.md.

5. Normal operator workflows

5.1 Start-of-day / before class

```
./scripts/azure/etis-azure status  
./scripts/azure/etis-azure health
```

- If both pass and there was no deployment/change, no deeper action is normally required.
- If status or health warns/fails, move to replicas and logs before changing anything.

5.2 After every production deployment

```
./scripts/azure/etis-azure acceptance
```

Controlled deployment and acceptance

Local CLI intentionally does not deploy production.



Figure 2. Deployment remains controlled in GitHub; the CLI validates the deployed result. Historical reference figure; current text governs.

5.3 Investigating a student report

1. Confirm whether the issue is one student/team or platform-wide.
2. Run `status` and `health`.
3. If platform health is good, use Studio/instructor workflows to inspect identity, roster, GitHub authorization, repository verification, or the specific review.
4. Use `logs` only after you know the approximate time/action being investigated.

5.4 Handling drift

Runtime drift response

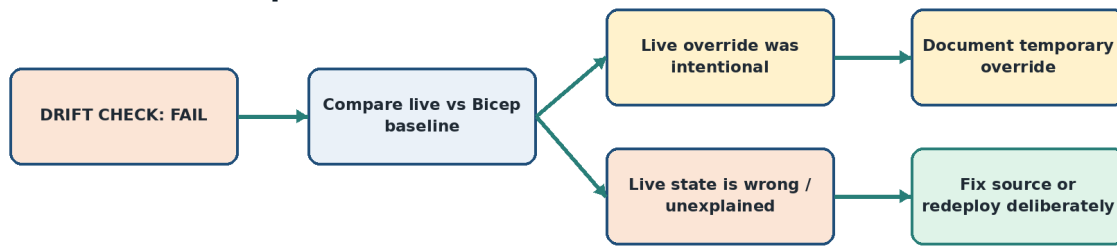


Figure 3. Drift is an investigation signal. Decide which side is authoritative before reconciling. Historical reference figure; current text governs.

6. Troubleshooting patterns

Symptom	Run next	Likely boundary
Site unavailable	status → health → replicas → logs	Container App/ingress/runtime
/health 200 but /ready fails	health → logs → config	Database, migration, startup dependency
Login loop/blank shell	smoke → logs	Entra callback/session/bootstrap
Repository verification fails	smoke/status then instructor/student onboarding checks	GitHub App authorization or wrong repo
Repeated restarts	replicas → logs	Application startup/resource/dependency
Unexpected 0 replicas	status → config → drift	Scaling/runtime drift
Wrong revision	status → revisions → GitHub workflow run	Deployment selection/traffic
High cost alert	budget → cost	Stewardship/capacity; not automatically health
Cost command 429	No immediate retry	Azure Cost Management throttling

7. Escalation and stop conditions

- Wrong Azure subscription/resource group/application context.
- `migration\_current=False` or schema head mismatch.
- Production environment reports development settings or non-production identity behavior.
- Smoke test shows authenticated application shell can flash before authentication state is established.
- GitHub App is scoped to All repositories.
- A proposed recovery requires editing production data/schema directly or exposing secrets.
- A runtime override is being considered as a substitute for understanding an application failure.

Appendix A. Quick command matrix

Command	Read-only?	Typical use
help	Yes	Display the command catalog and command-specific option pointers.
doctor	Yes	Check workstation tools, repository context, Azure authentication/subscription, Container App access, and source replica baseline.
status	Yes	Show a concise production snapshot.
config	Yes	Display important non-secret runtime configuration and source baseline.
health	Yes	Check public `/health`, `/ready`, migration-current state, root reachability, and latency.
replicas	Yes	Inspect configured capacity, replica readiness, restart count, revision ownership, and running state.
revisions	Yes	Inspect revision history and traffic allocation.
logs	Yes	Show structured recent console activity while suppressing routine probes by default.
cost	Yes	Show month-to-date Azure cost telemetry.
budget	Yes	Verify the production budget and thresholds without printing recipient email addresses.
smoke	Yes	Run non-destructive external production checks including authentication/bootstrap invariants.
drift	Yes	Compare runtime state with source-of-truth and accepted production invariants.
acceptance	Yes	Orchestrate the required post-deployment checks and issue the production decision.

Command	Read-only?	Typical use
scale	No	Apply a guarded live runtime min/max replica override.

Appendix B. Accepted baseline

Invariant	Accepted value
Application/resource group	etis-studio-prod
Public origin	https://simulator.etisframework.org
Revision mode	Single
Ingress	External HTTPS
Target port	8000
Min/max replicas	1 / 5
Budget	\$100 monthly; actual-cost alerts at 50%, 80%, 100%
Database	PostgreSQL Flexible Server; migration_current must be true
Deployment	Protected GitHub Deploy Azure workflow after merge to main

Appendix C. Representative healthy output

Exact latency, revision names, replica IDs, costs, and timestamps vary. The examples below show the shape and decision signals an operator should expect rather than values to copy literally.

status

```

ETIS Engineering Studio
Production Status
Environment ..... PRODUCTION
Application ..... etis-studio-prod
Resource Group ..... etis-studio-prod
Provisioning State ..... PASS Succeeded
Revision Mode ..... PASS Single
Traffic ..... PASS 100%
Min / Max Replicas ..... PASS 1 / 5
Running Replicas ..... PASS 1
STATUS: PASS

```

health

```

ETIS Engineering Studio
Application Health
/health ..... PASS  HTTP 200
  status=ok, environment=production
/ready ..... PASS  HTTP 200
  status=ready, migration_current=True
Root Page ..... PASS  HTTP 200
HEALTH: PASS

```

replicas

```

Configured Capacity ..... PASS  1 / 5
Running Replicas ..... PASS  1
Replica Revision Ready Restarts State
<replica-id> <latest-revision> Yes 0 Running
REPLICAS: PASS

```

budget

```

Budget Name ..... etis-studio-prod-monthly
Monthly Budget ..... PASS  $100
50% Actual Cost Alert ..... PASS  enabled
80% Actual Cost Alert ..... PASS  enabled
100% Actual Cost Alert ..... PASS  enabled
BUDGET: PASS

```

drift

```

Bicep Min Replicas ..... PASS  1
Runtime Min Replicas ..... PASS  1
Bicep Max Replicas ..... PASS  5
Runtime Max Replicas ..... PASS  5
Revision Mode ..... PASS  Single
External Ingress ..... PASS
Target Port ..... PASS  8000
Latest Revision Traffic ..... PASS  100%
Fail-Closed Shell ..... PASS
DRIFT CHECK: PASS

```

acceptance

```

Required Production Checks
  Production Status ..... PASS
  Application Health ..... PASS
  Replica Capacity ..... PASS
  External Smoke Test ..... PASS
  Runtime Drift ..... PASS
  Budget Controls ..... PASS
Informational Checks
  Cost Telemetry ..... PASS
PRODUCTION ACCEPTANCE: PASS

```

scale 1 5 (no-op)

```

Environment ..... PRODUCTION
Current Min / Max ..... 1 / 5
Requested Min / Max ..... 1 / 5
Change Required ..... PASS runtime already matches requested values
Result: No Azure change was made.
SCALE OVERRIDE: PASS

```

Appendix D. Exit behavior and automation

Situation	Expected behavior
Successful required check	Exit 0; final PASS.
Advisory condition designed not to block operation	Exit 0 with WARN when appropriate; caller must read status text.
Required failure or invalid operation	Non-zero exit; FAIL/error detail.
Invalid `scale` arguments	Rejected locally before Azure mutation/authentication where possible.
Cost Management throttling	COST: WARN; non-blocking because cost is informational.
Unknown command/option	Usage/error with non-zero exit.

Automation should test both process exit status and the documented final decision label when the distinction between required and advisory checks matters.

Appendix E. Temporary runtime override record

When `scale` is used for a real production change, record the override so a later operator does not mistake deliberate temporary state for unexplained drift.

Field	Record
Date/time and operator	
Reason / incident or change reference	
Previous min/max	
Requested min/max	
Expected duration	
Bicep/source changed?	No, unless separately approved and merged
Validation performed	status / health / drift / acceptance as appropriate
Reconciliation plan	When and how runtime will return to the authoritative baseline